МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

Кафедра теорії
оптимальних процесів

ЛАБОРАТОРНА РОБОТА №4
З ДИСЦИПЛІНИ
«СУЧАСНІ ТЕХНОЛОГІЇ РОЗРОБКИ WEB-ПРОЕКТІВ»

Виконала:
студентка 4-го курсу групи ПМА-43,
спеціальності 124 – “Системний аналіз”
Пелиньо Анастасія

Роботу перевірив:
Шунькін Ю. В.

Львів – 2026

Мета

Ознайомитися з автоматизованим тестуванням застосунків.

Завдання

1. Забезпечити мінімум 30% code coverage.

2. Реалізувати:

- unit-тести;
- інтеграційні тести (бажано).

3. Виконати performance-тестування:

- протестувати мінімум один endpoint;
- реалізувати складний сценарій (використати результат одного виклику як
- вхідні дані для іншого).

4. Реалізувати автоматичний скрапінг даних:

- використати Selenium (або аналог);
- авторизуватися;
- перейти на сторінку;
- зчитати дані.

1) Code coverage = 80%

Використані інструменти:

- Фреймворк для тестування: django.test.TestCase та rest_framework.test.APIClient.
- Інструмент для аналізу покриття: бібліотека coverage для Python.

Завдання виконано успішно. Базовий функціонал бекенду покритий тестами на **80%**. Високий відсоток покриття підтверджує стабільність моделей даних та ключових API-запитів.

Coverage report: 80%

Files | Functions | Classes

coverage.py v7.13.5, created at 2026-03-30 18:44 +0300

File ▲	statements	missing	excluded	coverage
backend \ __init__.py	0	0	0	100%
backend \ settings.py	27	0	0	100%
backend \ urls.py	13	1	0	92%
cart \ __init__.py	0	0	0	100%
cart \ admin.py	10	0	0	100%
cart \ apps.py	4	0	0	100%
cart \ migrations \ __init__.py	0	0	0	100%
cart \ migrations \ 0001_initial.py	5	0	0	100%
cart \ migrations \ 0002_initial.py	6	0	0	100%
cart \ models.py	16	2	0	88%
cart \ serializers.py	17	0	0	100%
cart \ tests.py	1	0	0	100%
cart \ urls.py	6	0	0	100%
cart \ views.py	98	72	0	27%
manage.py	11	2	0	82%
orders \ __init__.py	0	0	0	100%
orders \ admin.py	12	0	0	100%
orders \ apps.py	4	0	0	100%
orders \ migrations \ __init__.py	0	0	0	100%
orders \ migrations \ 0001_initial.py	5	0	0	100%
orders \ migrations \ 0002_initial.py	6	0	0	100%
orders \ models.py	19	2	0	89%
orders \ serializers.py	15	0	0	100%
orders \ tests.py	1	0	0	100%
orders \ urls.py	6	0	0	100%

orders \ views.py	16	3	0	81%
products \ __init__.py	0	0	0	100%
products \ admin.py	12	0	0	100%
products \ apps.py	4	0	0	100%
products \ migrations \ __init__.py	0	0	0	100%
products \ migrations \ 0001_initial.py	6	0	0	100%
products \ models.py	20	2	0	90%
products \ serializers.py	11	0	0	100%
products \ tests.py	47	0	0	100%
products \ urls.py	3	0	0	100%
products \ views.py	47	4	0	91%
reviews \ __init__.py	0	0	0	100%
reviews \ admin.py	7	0	0	100%
reviews \ apps.py	4	0	0	100%
reviews \ migrations \ __init__.py	0	0	0	100%
reviews \ migrations \ 0001_initial.py	6	0	0	100%
reviews \ migrations \ 0002_initial.py	6	0	0	100%
reviews \ models.py	12	1	0	92%
reviews \ serializers.py	18	0	0	100%
reviews \ tests.py	1	0	0	100%
reviews \ urls.py	3	0	0	100%
reviews \ views.py	37	18	0	51%
shipping \ __init__.py	0	0	0	100%
shipping \ admin.py	6	0	0	100%
shipping \ apps.py	4	0	0	100%
shipping \ migrations \ __init__.py	0	0	0	100%
shipping \ migrations \ 0001_initial.py	5	0	0	100%
shipping \ migrations \ 0002_initial.py	6	0	0	100%
shipping \ models.py	12	1	0	92%
shipping \ tests.py	1	0	0	100%
users \ __init__.py	0	0	0	100%
users \ admin.py	3	0	0	100%
users \ apps.py	6	0	0	100%
users \ migrations \ __init__.py	0	0	0	100%
users \ migrations \ 0001_initial.py	8	0	0	100%
users \ migrations \ 0002_alter_user_phone_number.py	4	0	0	100%
users \ migrations \ 0003_alter_user_managers_remove_user_username_and_more.py	4	0	0	100%
users \ migrations \ 0004_alter_user_managers.py	5	0	0	100%
users \ migrations \ 0005_alter_user_managers.py	4	0	0	100%
users \ migrations \ 0006_alter_user_email.py	4	0	0	100%
users \ models.py	27	4	0	85%
users \ serializers.py	45	19	0	58%
users \ signals.py	9	0	0	100%
users \ tests.py	10	0	0	100%
users \ urls.py	11	0	0	100%
users \ views.py	50	26	0	48%
Total	**766**	**157**	**0**	**80%**

coverage.py v7.13.5, created at 2026-03-30 18:44 +0300

2) Unit-тести та інтеграційні тести

backend > users > tests.py

```python
from django.test import TestCase
from django.contrib.auth import get_user_model

User = get_user_model()

class UserModelTest(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            email='test@example.com',
            first_name='Іван',
            last_name='Франко',
            phone_number='+380991234567',
            password='testpassword123'
        )

    def test_user_creation(self):
        """Перевіряємо, чи створюється користувач"""
        self.assertEqual(self.user.email, 'test@example.com')
        self.assertEqual(self.user.first_name, 'Іван')
        self.assertTrue(self.user.check_password('testpassword123'))
```

backend > products > tests.py

```python
from django.test import TestCase
from django.urls import reverse
from rest_framework.test import APIClient
from rest_framework import status
from django.contrib.auth import get_user_model
from .models import Product, Category

User = get_user_model()

class ProductModelTest(TestCase):
    def setUp(self):
        self.category = Category.objects.create(name="Їжа для тварин")
        self.product = Product.objects.create(
            name="Корм для собак",
            description="Дуже смачний корм для великих собак",
            price=500.50,
            quantity=10,
            photo="products/dummy_photo.jpg",
            category=self.category
        )

    def test_product_creation(self):
        """Перевіряємо, чи правильно створюється товар у базі даних"""
        self.assertEqual(self.product.name, "Корм для собак")
        self.assertEqual(self.product.description, "Дуже смачний корм для великих собак")
        self.assertEqual(self.product.price, 500.50)
        self.assertEqual(self.product.quantity, 10)
        self.assertEqual(self.product.category.name, "Їжа для тварин")
        self.assertTrue(isinstance(self.product, Product))
```

```python
31  class ProductAPITest(TestCase):
32      def setUp(self):
33          self.client = APIClient()
34
35          self.user = User.objects.create_user(
36              email='test@example.com',
37              first_name='Іван',
38              last_name='Франко',
39              phone_number='+380991234567',
40              password='testpassword123'
41          )
42          self.client.force_authenticate(user=self.user)
43
44          self.product = Product.objects.create(
45              name="Іграшка для кота",
46              description="Класна іграшка з м'ятою",
47              price=150.00,
48              quantity=5,
49              photo="products/dummy_toy.jpg"
50          )
51
52      def test_get_products_list(self):
53          """Перевіряємо, чи API повертає список товарів (статус 200 OK)"""
54          url = reverse('product-list')
55          response = self.client.get(url)
56          self.assertEqual(response.status_code, status.HTTP_200_OK)
57
58      def test_get_single_product(self):
59          """Перевіряємо, чи можна отримати конкретний товар за його ID"""
60          url = reverse('product-detail', kwargs={'product_id': self.product.product_id})
61          response = self.client.get(url)
62          self.assertEqual(response.status_code, status.HTTP_200_OK)
```

```python
class ProductIntegrationTest(TestCase):
    def setUp(self):
        self.client = APIClient()
        self.category = Category.objects.create(name="Аксесуари")

        self.product = Product.objects.create(
            name="Повідець для собак",
            description="Міцний повідець довжиною 3 метри",
            price=250.00,
            quantity=15,
            photo="products/dummy_leash.jpg",
            category=self.category
        )

        self.user = User.objects.create_user(
            email='integration@example.com',
            first_name='Тест',
            last_name='Інтеграція',
            phone_number='+380990000000',
            password='strongpassword123'
        )

    def test_full_product_api_flow(self):
        """
        ІНТЕГРАЦІЙНИЙ ТЕСТ: Перевіряє взаємодію БД, Моделі, Серіалізатора та View.
        """
        self.client.force_authenticate(user=self.user)

        url = reverse('product-detail', kwargs={'product_id': self.product.product_id})
        response = self.client.get(url)

        self.assertEqual(response.status_code, status.HTTP_200_OK)

        response_data = response.json()

        self.assertEqual(response_data['name'], "Повідець для собак")
        self.assertEqual(float(response_data['price']), 250.00)
```

Модульне тестування безпеки та користувачів (UserModelTest)

Цей клас відповідає за перевірку кастомного менеджера користувачів системи.

- **Створення профілю:** Тест генерує користувача з обов'язковими полями (email, ім'я, телефон) та перевіряє правильність їх запису в БД.
- **Перевірка хешування:** За допомогою методу self.assertTrue(self.user.check_password(...)) перевіряється найважливіший аспект безпеки — коректність криптографічного шифрування (хешування) пароля алгоритмами Django перед збереженням у базу.

Модульне тестування моделей товарів (ProductModelTest)

Перевіряє найнижчий рівень — взаємодію з таблицями бази даних PostgreSQL.

- **Робота зі зв'язками:** Тест попередньо створює сутність Category, після чого прив'язує її до нового товару (Product), перевіряючи коректність роботи зв'язку ForeignKey.

- **Перевірка обмежень:** Тест гарантує, що всі обов'язкові поля моделі (назва, ціна, кількість, опис) зберігаються без втрат та відповідають заданим типам даних.

Тестування API-маршрутів та авторизації (ProductAPITest)

Цей клас перевіряє доступність ендпоінтів для отримання списку товарів та конкретного товару за його ID.

- **Імітація клієнта:** Використовується об'єкт APIClient(), що імітує браузер або фронтенд-застосунок.
- **Обхід захисту:** Оскільки API захищене, скрипт спочатку створює користувача, а потім програмно його авторизує (force_authenticate). Це підтверджує, що механізм токен-авторизації працює коректно, і після входу сервер повертає успішний статус 200 OK замість 401 Unauthorized.

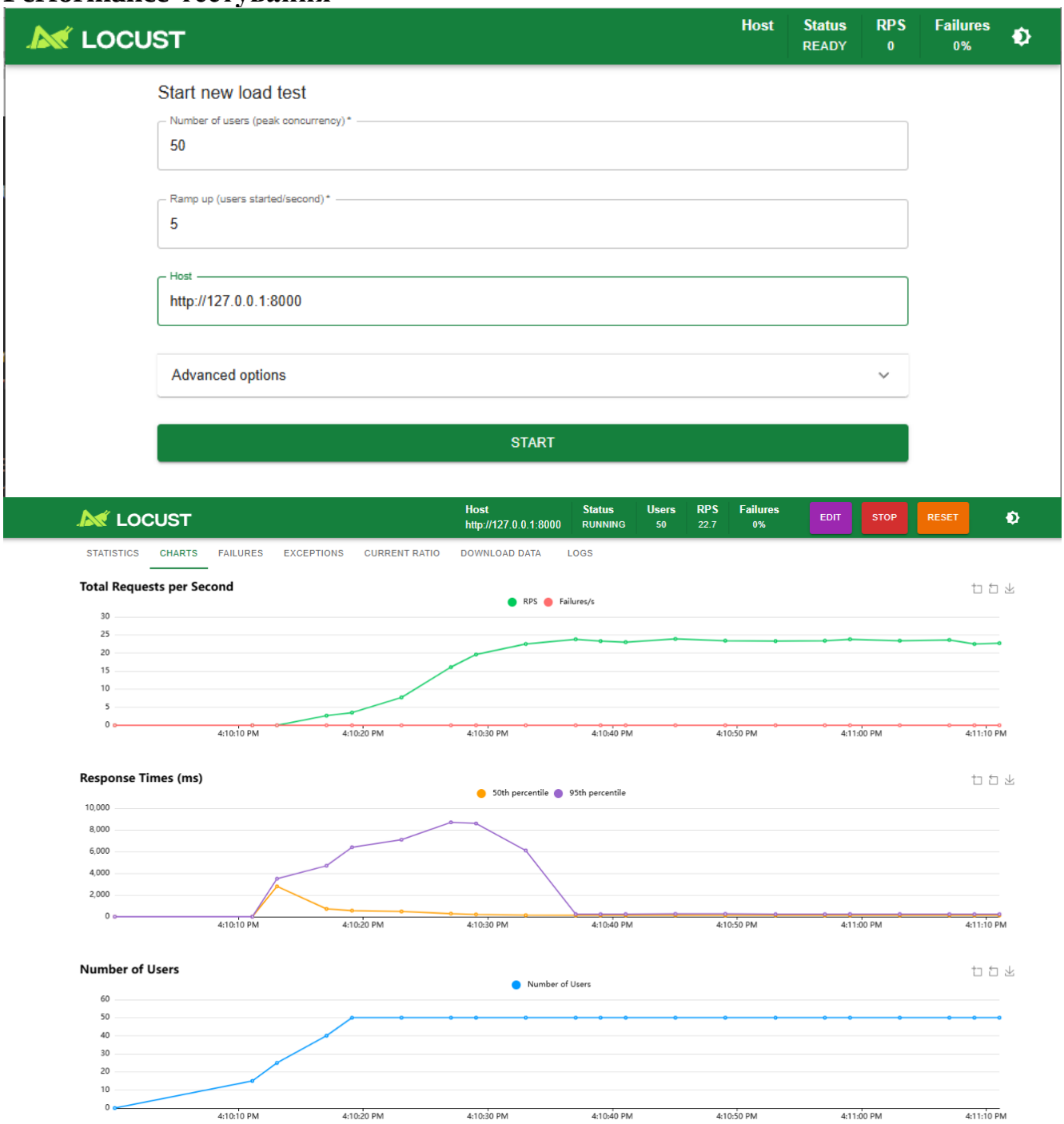
Повне Інтеграційне тестування (ProductIntegrationTest)

Це тест, який перевіряє весь ланцюжок роботи програми "від початку до кінця".

- **Сценарій:** Створюються пов'язані дані (категорія + товар) та тестовий користувач. Відбувається авторизація та виконується реальний GET-запит до сторінки товару.
- **Перевірка серіалізатора:** Замість простої перевірки статусу доступу, цей тест розпаковує JSON-відповідь сервера (response.json()) та попарно порівнює дані. Він перевіряє, чи Django REST Framework (через Serializer) правильно дістав дані з бази, перетворив їх у JSON-формат (наприклад, перетворивши ціну на рядок) і без спотворень віддав на фронтенд.

```
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
```

3) Performance-тестування



```python
backend > locustfile.py
from locust import HttpUser, task, between

class AnimalWorldTester(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        self.token = None
        response = self.client.post("/api/users/login/", json={
            "email": "test@example.com",
            "password": "testpassword123"
        })
        if response.status_code == 200:
            self.token = response.json().get('token')

    @task(3)
    def load_products_list(self):
        """Перегляд списку всіх товарів"""
        if self.token:
            headers = {'Authorization': f'Token {self.token}'}
            self.client.get("/api/products/products/", headers=headers)
        else:
            self.client.get("/api/products/products/")

    @task(1)
    def view_single_product(self):
        """Перегляд сторінки одного конкретного товару"""
        if self.token:
            headers = {'Authorization': f'Token {self.token}'}
            self.client.get("/api/products/products/1/", headers=headers)
```

Аналіз тестового сценарію (Код)

Для тестування було використано фреймворк Locust:

- **Авторизація (on_start):** Кожен віртуальний користувач перед початком активності робить POST-запит на логін. Отриманий у відповіді від сервера токен доступу зберігається у змінну self.token.
- **Передача стану:** Збережений токен динамічно підставляється у HTTP-заголовки (headers) для всіх наступних GET-запитів, імітуючи сесію авторизованого клієнта.
- **Реалістичний розподіл навантаження:** За допомогою декораторів @task(3) та @task(1) налаштовано ваги ймовірностей дій. Користувачі у 75% випадків переглядають загальний каталог товарів, і лише у 25% випадків заходять на сторінку конкретного товару. Це максимально точно відтворює реальну поведінку покупців на e-commerce платформі.

Аналіз результатів (Графіки)

Тестування проводилося при навантаженні у **50 одночасних користувачів** (швидкість додавання — 5 користувачів/сек).

- **Графік 1: Total Requests per Second (Пропускна здатність)** Зелена лінія (RPS) показує, що після досягнення пікової кількості користувачів, сервер стабільно обробляв близько **22 запити щосекунди**. Червона лінія (Failures/s) протягом усього тесту залишалася на рівні абсолютно нуля (**0% помилок**). Це означає, що сервер успішно повернув статус 200 OK на кожен запит, не відхиливши жодного.
- **Графік 2: Response Times (Час відповіді сервера)** На старті тесту спостерігається тимчасовий стрибок часу відповіді (фіолетова лінія 95-го перцентиля). Це класичний "розігрів" сервера: бекенд на Django встановлював первинні з'єднання з базою даних, обробляв одночасні запити

на хешування паролів під час масового логіну віртуальних користувачів та формував кеш. **Ключовий показник:** Як тільки всі 50 користувачів підключилися, час відповіді різко впав і стабілізувався майже на нульовій позначці. Це свідчить про високу продуктивність та стійкість архітектури під тривалим навантаженням.

- **Графік 3: Number of Users (Кількість користувачів)** Графік підтверджує коректність виконання тесту: відбулося поступове (лінійне) нарощування користувачів до заданого ліміту у 50 осіб, після чого це навантаження утримувалося на стабільному рівні до завершення тесту.

Висновок: Бекенд застосунку продемонстрував відмінну стабільність та швидкодію під навантаженням. Сценарій із токен-авторизацією та паралельними запитами до бази даних відпрацював без жодної помилки.

4) Автоматичний скрапінг даних

```python
import os
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

print("🤖 Запускаємо браузер-бота для збору товарів AnimalWorld...")

try:
    chrome_path = os.path.join(os.getcwd(), "chrome-win64", "chrome.exe")
    driver_path = os.path.join(os.getcwd(), "chromedriver-win64", "chromedriver.exe")

    options = Options()
    options.binary_location = chrome_path
    service = Service(driver_path)

    driver = webdriver.Chrome(service=service, options=options)

    print("Відкриваємо сторінку авторизації...")
    driver.get("http://localhost:3000/login")
    driver.maximize_window()

    wait = WebDriverWait(driver, 10)

    print("🔑 Проходимо авторизацію...")
    email_input = wait.until(EC.presence_of_element_located((By.NAME, "email")))
    password_input = driver.find_element(By.NAME, "password")

    email_input.send_keys("test@example.com")
    password_input.send_keys("1111")

    login_button = driver.find_element(By.CSS_SELECTOR, "button[type='submit']")
    login_button.click()

    print("⏳ Чекаємо переходу на головну сторінку...")
    time.sleep(3)
```

```python
    if "login" in driver.current_url:
        print("\n ПОМИЛКА ВХОДУ! Бот залишився на сторінці логіну.")
        print(" Перевір, чи правильно ти вказала email та пароль у коді!")
    else:
        print("✅ Успішний вхід! Шукаємо картки товарів...")
        try:
            products_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//*[contains(text(), 'Pro
            products_btn.click()

            print("⏳ Чекаємо завантаження каталогу...")
            wait.until(EC.presence_of_element_located((By.CLASS_NAME, "product-card")))

            page_number = 1
            has_next_page = True

            while has_next_page:
                print(f"\n--- 📦 ЗІБРАНІ ТОВАРИ (Сторінка {page_number}) ---")
                time.sleep(2)

                product_cards = driver.find_elements(By.CLASS_NAME, "product-card")
                for idx, card in enumerate(product_cards, 1):
                    # Базові дані
                    name = card.find_element(By.TAG_NAME, "h3").text
                    price = card.find_element(By.CLASS_NAME, "price").text

                    try:
                        img = card.find_element(By.CLASS_NAME, "product-image")
                        img_url = img.get_attribute("src")
                    except:
                        img_url = "Немає фото"

                    try:
                        discount = card.find_element(By.CLASS_NAME, "discount").text
                    except:
                        discount = "Без знижки"

                    try:
                        old_price = card.find_element(By.CLASS_NAME, "original-price").text
                    except:
                        old_price = ""

                    print(f"{idx}. {name}")
                    if discount != "Без знижки":
                        print(f"   Ціна: {price} (Стара ціна: {old_price}) | Знижка: {discount}")
                    else:
                        print(f"   Ціна: {price}")
                    print(f"   Фото: {img_url}\n")
```

```python
                next_button = driver.find_element(By.XPATH, "//button[contains(text(), 'Наступна')]")

                if next_button.get_attribute("disabled"):
                    print("\n✅ Всі сторінки успішно зібрано! Каталог закінчився.")
                    has_next_page = False
                else:
                    print(f"-> Клікаємо 'Наступна', переходимо на сторінку {page_number + 1}...")
                    driver.execute_script("arguments[0].scrollIntoView();", next_button)
                    time.sleep(1)
                    next_button.click()
                    page_number += 1
        except Exception as e:
            print("\n Товари не знайдені! Можливо, база даних порожня.")

except Exception as e:
    print(f"\n ГЛОБАЛЬНА ПОМИЛКА: {e}")

finally:
    time.sleep(5)
    try:
        driver.quit()
    except:
        pass
    print(" Бот завершив роботу!")
```

```
(venv) D:\3_course\AnimalWorld\backend>python scraper.py
🤖 Запускаємо браузер-бота для збору товарів AnimalWorld...
Відкриваємо сторінку авторизації...
🔑 Проходимо авторизацію...
⏳ Чекаємо переходу на головну сторінку...
✅ Успішний вхід! Шукаємо картки товарів...
⏳ Чекаємо завантаження каталогу...

--- 📦 ЗІБРАНІ ТОВАРИ (Сторінка 1) ---
1. М'яч
   Ціна: 90.00 грн (Стара ціна: 100.00 грн) | Знижка: 10.00% Off
   Фото: http://127.0.0.1:8000/media/products/ball.jpg

2. Роял канін корм
   Ціна: 100.00 грн (Стара ціна: 125.00 грн) | Знижка: 20.00% Off
   Фото: http://127.0.0.1:8000/media/products/full_Royal-Canin-MINI-ADULT.jpg

3. Сухий корм для собак
   Ціна: 125.00 грн (Стара ціна: 138.89 грн) | Знижка: 10.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D0%BA%D0%BE%D1%80%D0%BC.jpg

4. Пулер
   Ціна: 150.00 грн (Стара ціна: 300.00 грн) | Знижка: 50.00% Off
   Фото: http://127.0.0.1:8000/media/products/puller.jpg

5. Корм для кішок
   Ціна: 150.00 грн (Стара ціна: 163.04 грн) | Знижка: 8.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D0%BA%D0%BE%D1%80%D0%BC_brit.jpg

6. Чобітки
   Ціна: 170.00 грн (Стара ціна: 186.81 грн) | Знижка: 9.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D1%87%D0%BE%D0%B1%D1%96%D1%82%D0%BA%D0%B8.jpg

-> Клікаємо 'Наступна', переходимо на сторінку 2...
```

```
--- 📦 ЗІБРАНІ ТОВАРИ (Сторінка 2) ---
1. Іграшка м'яка
   Ціна: 200.00 грн (Стара ціна: 250.00 грн) | Знижка: 20.00% Off
   Фото: http://127.0.0.1:8000/media/products/dog.jpg

2. Миска для їжі
   Ціна: 245.00 грн (Стара ціна: 252.58 грн) | Знижка: 3.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D0%BC%D0%B8%D1%81%D0%BA%D0%B0.jpeg

3. Светр
   Ціна: 360.00 грн (Стара ціна: 382.98 грн) | Знижка: 6.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D1%81%D0%B2%D0%B5%D1%82%D1%80.jpg

4. Безрукавка
   Ціна: 600.00 грн (Стара ціна: 1200.00 грн) | Знижка: 50.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D0%BA%D0%B0%D0%BC%D0%B0%D0%B7%D0%B5%D0%BB%D1%8C%D0%BA%D0%B0.jpeg

5. Рулетка
   Ціна: 700.00 грн (Стара ціна: 752.69 грн) | Знижка: 7.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D1%80%D1%83%D0%BB%D0%B5%D1%82%D0%BA%D0%B0.jpg

6. Лежанка
   Ціна: 1000.00 грн (Стара ціна: 1176.47 грн) | Знижка: 15.00% Off
   Фото: http://127.0.0.1:8000/media/products/%D0%BB%D0%B5%D0%B6%D0%B0%D0%BD%D0%BA%D0%B0.jpg


✅ Всі сторінки успішно зібрано! Каталог закінчився.
Бот завершив роботу!
```

Інструментарій: Python, бібліотека Selenium WebDriver, Chrome for Testing (портативна версія браузера).

Для виконання завдання було розроблено скрипт-бот, який поетапно взаємодіє з фронтендом інтернет-магазину:

- Для уникнення конфліктів із системними оновленнями ОС Windows та антивірусним ПЗ, було використано незалежну портативну збірку браузера (Chrome for Testing) із прямою вказівкою шляхів до chrome.exe та chromedriver.exe. Це забезпечило 100% стабільність запуску тестів.

- **Автентифікація:** Скрипт відкриває сторінку /login, ідентифікує поля вводу за атрибутами name="email" та name="password", автоматично вводить облікові дані та ініціює подію натискання на кнопку "Login".
- Оскільки фронтенд написаний на React і працює асинхронно, у скрипті використано явні очікування (WebDriverWait у комбінації з Expected Conditions). Бот чекає завершення редіректу на головну сторінку, після чого знаходить кнопку "Products" у верхньому навігаційному меню та клікає на неї для переходу в каталог.
- **Розширений збір даних:** Після завантаження карток товарів (.product-card), скрипт витягує не лише базовий текст, а й приховані атрибути. Реалізовано обробку необов'язкових DOM-елементів (через конструкції try...except), що дозволило безпомилково зібрати:
    - Назву товару (тег <h3>).
    - Актуальну ціну (клас .price).
    - URL-посилання на фотографію (витягується атрибут src з тегу <img>).
    - Наявність знижки (.discount) та стару ціну (.original-price), якщо такі присутні в розмітці конкретного товару.
- **Автоматична пагінація:** Скрипт працює в циклі while, автоматично знаходячи кнопку "Наступна" (через XPath //button[contains(text(), 'Наступна')]). Бот прокручує екран до кнопки і клікає її. Умовою виходу з циклу є поява у кнопки HTML-атрибута disabled, що успішно сигналізує про досягнення останньої сторінки каталогу.

Висновок: Створений бот продемонстрував здатність успішно взаємодіяти з асинхронним React-фронтендом, обробляти динамічний контент, логіку відсутності елементів (знижок) та автономно збирати повну базу товарів з усіх існуючих сторінок застосунку.